Chapter 8: Sorting in Linear Time

Chapter 8 overview

How fast can we sort?

We will prove a lower bound, then beat it by playing a different game.

Comparison sorting

- The only operation that may be used to gain order information about a sequence is comparison of pairs of elements.
- All sorts seen so far are comparison sorts: insertion sort, selection sort, merge sort, quicksort, heapsort, treesort.

Lower bounds for sorting

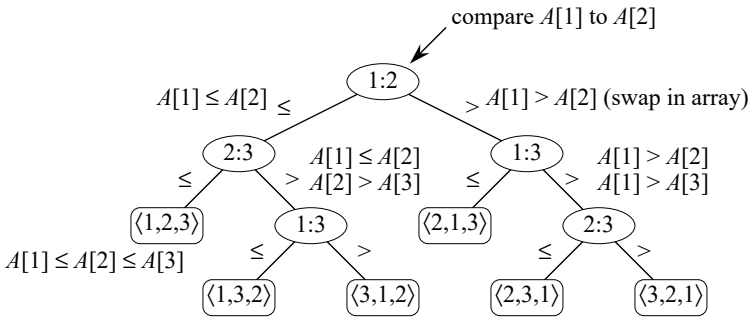
Lower bounds

- $\Omega(n)$ to examine all the input.
- All sorts seen so far are $\Omega(n \lg n)$.
- We'll show that $\Omega(n \lg n)$ is a lower bound for comparison sorts.

Decision tree

- Abstraction of any comparison sort.
- Represents comparisons made by
  - a specific sorting algorithm
  - on inputs of a given size.
- Abstracts away everything else: control and data movement.
- We're counting *only* comparisons.

For insertion sort on 3 elements:



[Each internal node is labeled by indices of array elements ***from their original positions****. Each leaf is labeled by the permutation of orders that the algorithm determines.]*

How many leaves on the decision tree? There are $\geq n!$ leaves, because every permutation appears at least once.

For any comparison sort,

- 1 tree for each n.
- View the tree as if the algorithm splits in two at each node, based on the information it has determined up to that point.
- The tree models all possible execution traces.

What is the length of the longest path from root to leaf? The worst-case number of comparisons

- Depends on the algorithm
- Insertion sort: $\Theta(n^2)$
- Merge sort: $\Theta(n \lg n)$

Theorem

Any decision tree that sorts n elements has height $\Omega(n \lg n)$.

Proof Let the decision tree have height h and l reachable leaves.

- $l \geq n!$
- $l \leq 2^h$ (see Section B.5.3: in a complete k-ary tree, there are k^h leaves) You can prove this with Mathematical induction
- $n! \leq l \leq 2^h$ or $2^h \geq n!$
- Take logarithms: $h \geq \lg(n!)$
- Use Stirling's approximation: $n! > (n/e)^n$ (by equation (3.23))

$$\begin{aligned} h &\geq \lg(n/e)^n \\ &= n \lg(n/e) \\ &= n \lg n - n \lg e \\ &= \Omega(n \lg n) \,. \end{aligned}$$

■ (theorem)

Corollary

Heapsort and merge sort are asymptotically optimal comparison sorts.

Sorting in linear time

Non-comparison sorts.

Counting sort

Depends on a *key assumption*: numbers to be sorted are integers in $\{0, 1, \ldots, k\}$.

Input: $A[1:n]$, where $A[j] \in \{0, 1, \ldots, k\}$ for $j = 1, 2, \ldots, n$. Array A and values n and k are given as parameters.

Output: $B[1:n]$, sorted.

Auxiliary storage: $C[0:k]$

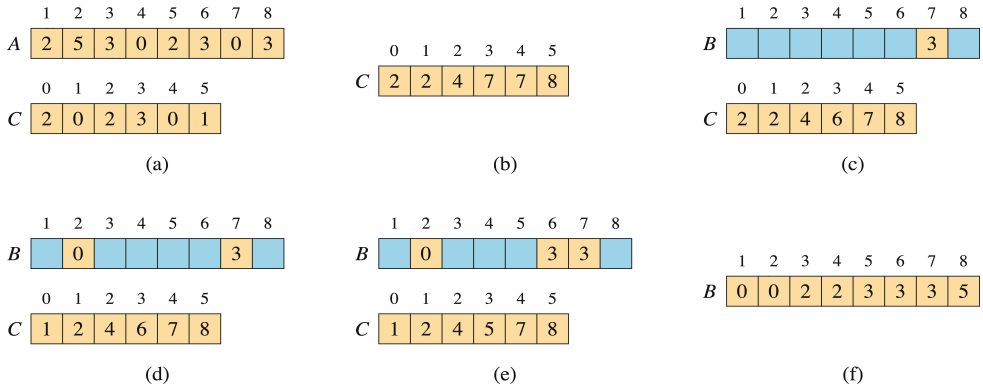
```
COUNTING-SORT(A, n, k)
  let B[1:n] and C[0:k] be new arrays
  for i = 0 to k
      C[i] = 0
  for j = 1 to n
      C[A[j]] = C[A[j]] + 1
  // C[i] now contains the number of elements equal to i.
  for i = 1 to k
      C[i] = C[i] + C[i − 1]
  // C[i] now contains the number of elements less than or equal to i.
  // Copy A to B, starting from the end of A.
  for j = n downto 1
      B[C[A[j]]] = A[j]
      C[A[j]] = C[A[j]] − 1    // to handle duplicate values
  return B
```

Example for $A = \langle 2_1, 5_1, 3_1, 0_1, 2_2, 3_2, 0_2, 3_3 \rangle$. *[Subscripts show original order of equal keys in order to demonstrate stability.]*

i	0	1	2	3	4	5
$C[i]$ after second **for** loop	2	0	2	3	0	1
$C[i]$ after third **for** loop	2	2	4	7	7	8

Sorted output is $\langle 0_1, 0_2, 2_1, 2_2, 3_1, 3_2, 3_3, 5_1 \rangle$.

Idea: After the third **for** loop, $C[i]$ counts how many keys are less than or equal to i. If all elements are distinct, then an element with value i should go into $B[i]$. But if elements are not distinct, by examining values in A in reverse order, the last **for** loop puts $A[j]$ into $B[C[A[j]]]$ and then decrements $C[A[j]]$ so that the next time it finds element with the same value as $A[j]$, that element goes into the position of B just before $A[j]$.



Loop invariant: At the start of each iteration of the last **for** loop, the last element in A with value i that has not yet been copied into B belongs in $B[C[i]]$.

Counting sort is ***stable*** (keys with same value appear in same order in output as they did in input) because of how the last loop works.

Analysis

$\Theta(n + k)$, which is $\Theta(n)$ if $k = O(n)$.

How big a k is practical?

- Good for sorting 32-bit values? No.
- 16-bit? Probably not.
- 8-bit? Maybe, depending on n.
- 4-bit? Probably (unless n is really small).

Counting sort will be used in radix sort.

Radix sort

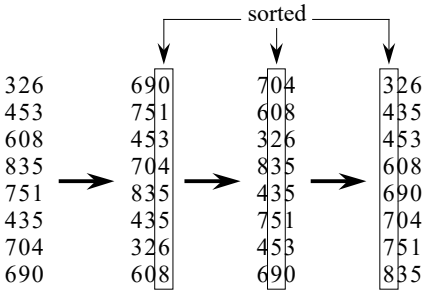
How IBM made its money. IBM made punch card readers for census tabulation in early 1900's. Card sorters worked on one column at a time. It's the algorithm for using the machine that extends the technique to multi-column sorting. The human operator was part of the algorithm!

Key idea: Sort *least* significant digits first.

To sort d digits:

```
RADIX-SORT(A, n, d)
  for i = 1 to d
      use a stable sort to sort array A[1 : n] on digit i
```

Example



Correctness

- Induction on number of passes (i in pseudocode).
- Assume digits $1, 2, \ldots, i - 1$ are sorted.
- Show that a stable sort on digit i leaves digits $1, \ldots, i$ sorted:
  - If two digits in position i are different, ordering by position i is correct, and positions $1, \ldots, i - 1$ are irrelevant.

- If two digits in position i are equal, the numbers are already in the right order (by inductive hypothesis). The stable sort on digit i leaves them in the right order.

This argument shows why it's so important to use a stable sort for intermediate sort.

Analysis

Assume that we use counting sort as the intermediate sort.

- $\Theta(n + k)$ per pass (digits in range $0, \ldots, k$)
- d passes
- $\Theta(d(n + k))$ total
- If $k = O(n)$, time $= \Theta(dn)$.

How to break each key into digits?

- n words.
- b bits/word.
- Break into r-bit digits. Have $d = \lceil b/r \rceil$.
- Use counting sort, $k = 2^r - 1$.

  Example: 32-bit words, 8-bit digits. $b = 32$, $r = 8$, $d = \lceil 32/8 \rceil = 4$, $k = 2^8 - 1 = 255$.
- Time $= \Theta((b/r)(n + 2^r))$.

How to choose r? Balance b/r and $n + 2^r$: decreasing r causes b/r to increase, but increasing r causes 2^r to increase.

If $b < \lfloor \lg n \rfloor$, then choose $r = b \Rightarrow (b/r)(n + 2^r) = \Theta(n)$, which is optimal.

If $b \geq \lfloor \lg n \rfloor$, then choosing $r \approx \lg n$ gives $\Theta((b/\lg n)(n + n)) = \Theta(bn/\lg n)$.

- Choosing $r < \lg n \Rightarrow b/r > b/\lg n$, and $n + 2^r$ term doesn't improve.
- Choosing $r > \lg n \Rightarrow n + 2^r$ term gets big. Example: $r = 2\lg n \Rightarrow 2^r = 2^{2\lg n} = (2^{\lg n})^2 = n^2$.

So, to sort 2^{16} 32-bit numbers, use $r = \lg 2^{16} = 16$ bits. $\lceil b/r \rceil = 2$ passes.

Compare radix sort to merge sort and quicksort:

- 1 million (2^{20}) 32-bit integers.
- Radix sort: $\lceil 32/20 \rceil = 2$ passes.
- Merge sort/quicksort: $\lg n = 20$ passes.
- Remember, though, that each radix sort "pass" is really 2 passes—one to take census, and one to move data.

How does radix sort violate the ground rules for a comparison sort?

- Using counting sort allows us to gain information about keys by means other than directly comparing two keys.
- Used keys as array indices.

Bucket sort

Assumes that the input is generated by a random process that distributes elements uniformly and independently over $[0, 1)$.

Idea

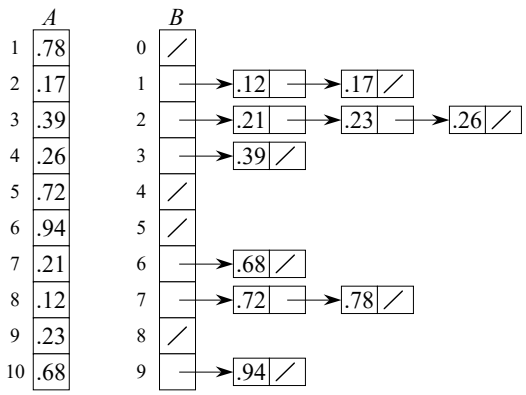
- Divide $[0, 1)$ into n equal-sized *buckets*.
- Distribute the n input values into the buckets. *[Can implement the buckets with linked lists; see Section 10.2.]*
- Sort each bucket.
- Then go through buckets in order, listing elements in each one.

Input: $A[1:n]$, where $0 \le A[i] < 1$ for all i.

Auxiliary array: $B[0:n-1]$ of linked lists, each list initially empty.

```
BUCKET-SORT(A, n)
  let B[0:n − 1] be a new array
  for i = 0 to n − 1
      make B[i] an empty list
  for i = 1 to n
      insert A[i] into list B[⌊n · A[i]⌋]
  for i = 0 to n − 1
      sort list B[i] with insertion sort
  concatenate lists B[0], B[1], . . . , B[n − 1] together in order
  return the concatenated lists
```

Example



[The buckets are shown after each has been sorted. Slashes indicate the end of each bucket.]

Correctness

Consider $A[i]$, $A[j]$. Assume without loss of generality that $A[i] \le A[j]$. Then $\lfloor n \cdot A[i] \rfloor \le \lfloor n \cdot A[j] \rfloor$. So $A[i]$ is placed into the same bucket as $A[j]$ or into a bucket with a lower index.

- If same bucket, insertion sort fixes up.
- If earlier bucket, concatenation of lists fixes up.

Analysis

- Relies on no bucket getting too many values.
- All lines of algorithm except insertion sorting take $\Theta(n)$ altogether.
- Intuitively, if each bucket gets a constant number of elements, it takes $O(1)$ time to sort each bucket $\Rightarrow$ $O(n)$ sort time for all buckets.
- We "expect" each bucket to have few elements, since the average is 1 element per bucket.
- But we need to do a careful analysis.

Define a random variable:

n_i = the number of elements placed in bucket $B[i]$.

Because insertion sort runs in quadratic time, bucket sort time is

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \ .$$

Take expectations of both sides:

$$\begin{aligned} \mathrm{E}[T(n)] &= \mathrm{E}\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} \mathrm{E}\left[O(n_i^2)\right] \quad \text{(linearity of expectation)} \\ &= \Theta(n) + \sum_{i=0}^{n-1} O(\mathrm{E}\left[n_i^2\right]) \quad (\mathrm{E}[aX] = a\mathrm{E}[X]) \end{aligned}$$

Claim

$\mathrm{E}[n_i^2] = 2 - (1/n)$ for $i = 0, \ldots, n-1$.

Proof is not covered.

Therefore:

$$\begin{aligned} \mathrm{E}[T(n)] &= \Theta(n) + \sum_{i=0}^{n-1} O(2 - 1/n) \\ &= \Theta(n) + O(n) \\ &= \Theta(n) \end{aligned}$$

n(2−1/n)=2n-1

- Again, not a comparison sort. Used a function of key values to index into an array.
- This is a ***probabilistic analysis***—we used probability to analyze an algorithm whose running time depends on the distribution of inputs.
- Different from a ***randomized algorithm***, where we use randomization to *impose* a distribution.
- With bucket sort, if the input isn't drawn from a uniform distribution on $[0, 1)$, the algorithm is still correct, but might not run in $\Theta(n)$ time. It runs in linear time as long as the sum of squares of bucket sizes is $\Theta(n)$.